# OUTSYSTEMS NOTE

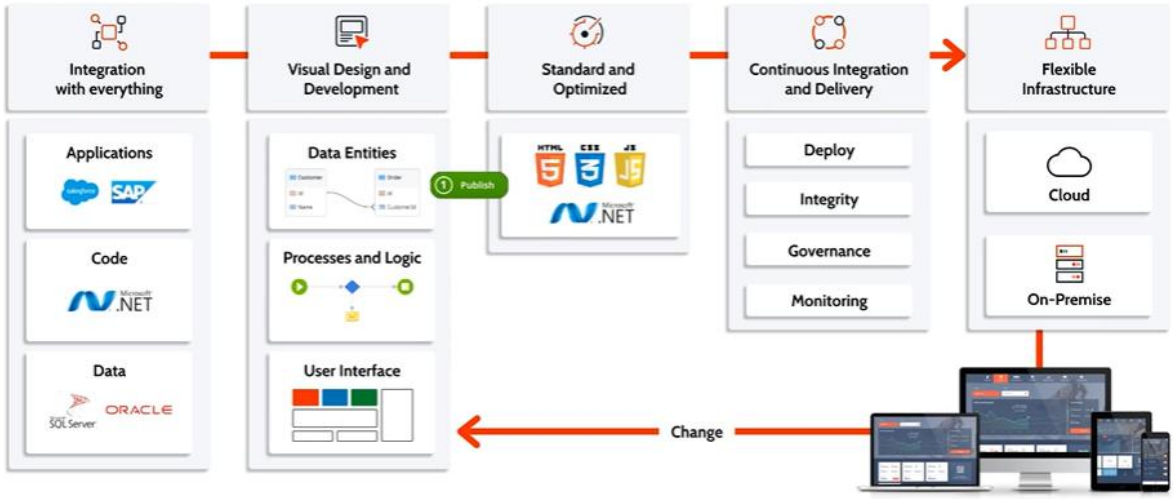
OutSystems là gì ?

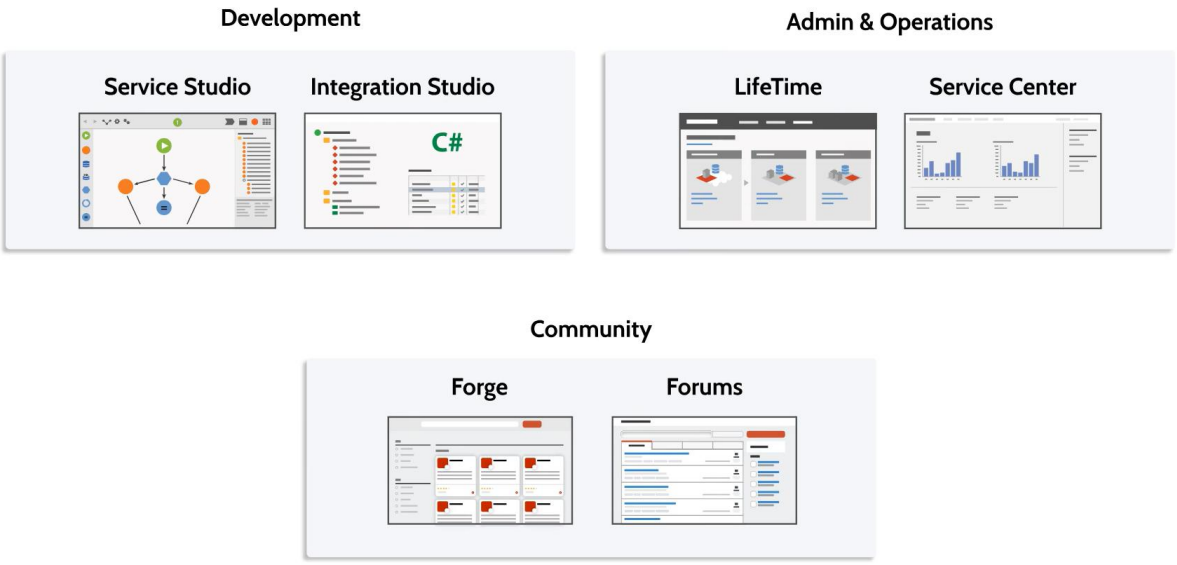
OutSystems hỗ trợ quá trình phát triển (development) và chuyển giao (delivery) các ứng dụng web (web applications) và ứng dụng di động (mobile applications) cấp doanh nghiệp (enterprise).

- Môi trường phát triển Low-code
- Tiêu chuẩn doanh nghiệp (enterprise-grade), full stack system
- Tích hợp (integrates) với các hệ thống khác (other systems)
- Khả năng quản lý (management) và phân tích (analytics) được tích hợp sẵn (built-in)

Các khả năng:

- Integration with everything: Salesforce, SAP, .Net, Oracle, SQL Server,…
- Using service studio – Visual design and Development
- Chuẩn hoá (standard) và optimized
- Integration và Delivery liên tục
- Cơ sở hạ tầng linh hoạt (Flexible Infrastructure): cloud, on-premise






Component and Tools là gì ?

- Platform Server: một tập hợp các máy chủ (set of servers) thực hiện nhiệm vụ biên dịch (compile), triển khai (deploy), quản lý (manage), vận hành (run) và giám sát (monitor).
- Service Studio: để create các ứng dụng di động (mobile applications) và ứng dụng web (web applications) tối ưu
- Integration Studio: Tích hợp với các External systems (.Net Code, Database)
- Service center: Cho phép review và cấu hình (configure) dưới góc độ quản trị (administration standpoint) và vận hành (operations standpoint)
- LifeTime: Quản lý vòng đời của các ứng dụng
- OutSystems Forge: https://www.outsystems.com/forge

---

Layer in Workspace OutSystems

- Procces
- Interface
- Logic
- Data

---

Proccess Layer

- Proccesses

    - Business Proccesses
    - Human Tasks
    - Automated Tasks
    - Decisions
    - Events
    - Waits
- Timer
    - Scheduled Actions
    - Priorites
    - Timeouts

---

Interface Layer

- UI Flows
    - Screens
    - Patterns/Block
- Images
    - Graphics
    - Icon
- Themes
    - Base look and feel
- Scripts

---

Logic Layer

- Client Actions
- Server Actions: SOAP Web Services, RESTful Services
- Integrations
- Roles
- Exceptions

---

Data Layer

- Entity Diagrams

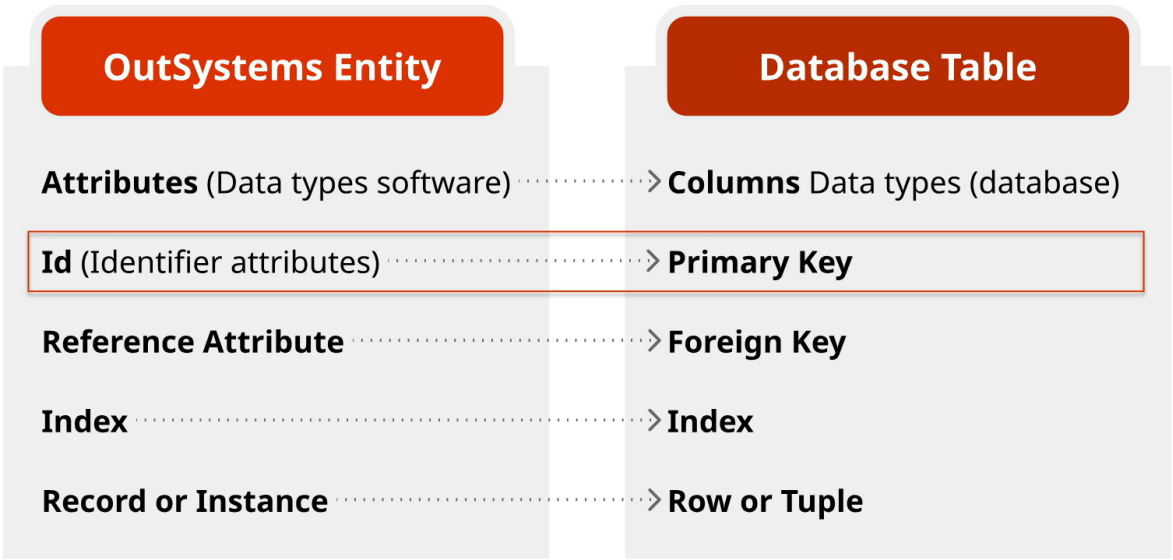
- Entities: Database, Local Storage
- Structures: Compound data types, In-memory data
- Client Variables
- Site Properties
- Resources

---

Modeling Data – Mô hình hoá dữ liệu

- Trong OutSystems, các khái niệm này được mô hình hóa (modeled) và được gọi là các thực thể (Entities).
- Các thực thể cơ sở dữ liệu (Database Entities) lưu giữ dữ liệu có thể được ghi/lưu trữ vào (stored in) và truy xuất từ (retrieved from) cơ sở dữ liệu (database).

Entity to Database Mapping




Entity là gì ?

Entity là một phần tử (element) cho phép chúng ta lưu trữ bền vững (persist) và truy xuất (access) các thông tin (information) cần thiết trong các applications của mình

Data Type

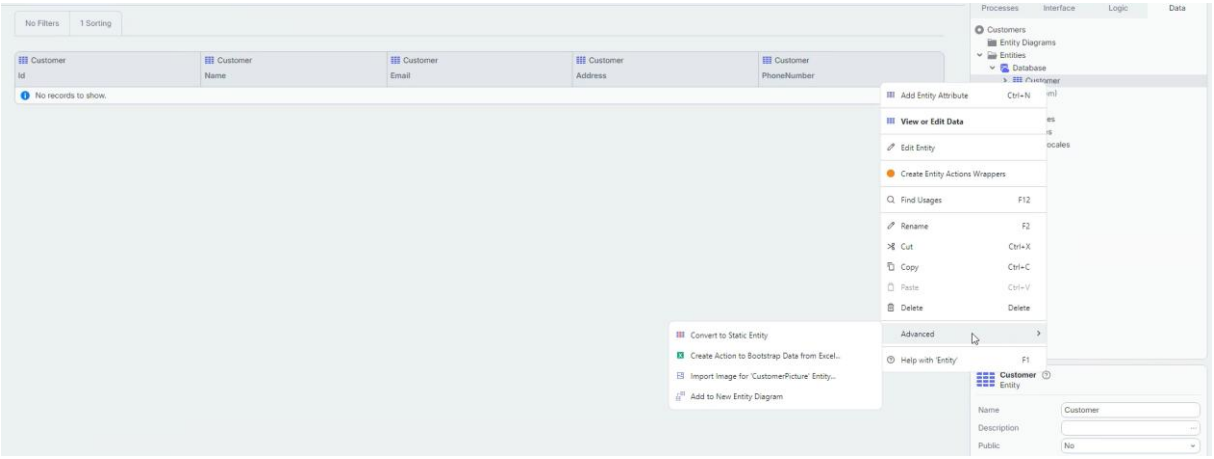
- Alphanumeric: Text, Phone Number, Email
- Numeric: Integer, Long integer, Decimal, Currency

- Dates and Times: Date Time, Date, Time
- Logic: Boolean
- Large Object: Binary Data
- Referential: Entity Identifier

---

Cách đẩy dữ liệu từ Excel sang thành một Entity

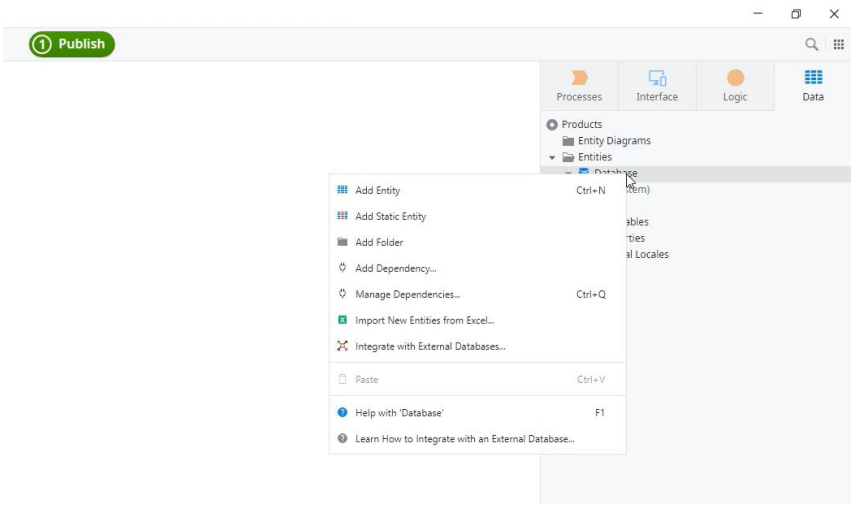
- Kiểm tra dữ liệu của một Entity → Chuột phải vào Entity → Chọn "View or edit data"
- Chuột phải vào Entity → Advanced → "Create Action to Bootstrap Data from Excel" → Publish




---

Cách tạo Entities và đẩy dữ liệu từ Excel

- Chuột phải vào mục Database → Chọn "Import New Entities from Excel" → Publish




---

Static Entities là gì ?

Static Entities là một loại thực thể (Entity) đặc biệt dùng để tạo ra một danh sách giá trị được định nghĩa trước (predefined list of values) có thể sử dụng trong các ứng dụng (applications) của chúng ta.

- Hoạt động như các kiểu liệt kê Enumerations/enum
- Bao gồm các Attributes và Record được định nghĩa trong giai đoạn design và development, không thể thay đổi trong runtime
- Chỉ có duy nhất 1 Entity Action

---

Entities Relationship

**2. Which of the following steps is necessary to create a 1-to-1 relationship between Entity A and Entity B?** + 0

- (●) Set the data type of the identifier attribute of Entity B to Entity A Identifier.
- ( ) Add a new Entity C, with two reference attributes of type Entity A Identifier and Entity B Identifier.
- ( ) Add a new reference attribute of type Entity B Identifier to Entity A.
- ( ) Add a new Entity C, with the identifier attribute being a composition of types Entity A Identifier and Entity B Identifier.

**3. Which of the following steps is necessary to create a 1-to-many relationship between a Master Entity A and a Detail Entity B?** + 0

- Set the data type of the identifier attribute of Entity B to Entity A Identifier.
- Add a new Entity C, with two reference attributes of type Entity A Identifier and Entity B Identifier.
- Entity A must have a reference attribute of type Entity B Identifier.
- (◉) Entity B must have a reference attribute of type Entity A Identifier.

**4. Which of the following steps is necessary to create a many-to-many relationship between Entity A and Entity B?** + 1

- Set the data type of the identifier attribute of Entity B to Entity A Identifier.
- Add a new reference attribute of type Entity B Identifier to Entity A.
- (◉) Add a new Entity C, with two reference attributes of type Entity A Identifier and Entity B Identifier.
- Add a new reference attribute of type Entity B Identifier to Entity A and a new reference attribute of type Entity A Identifier to Entity B.

---

Data model integrity

- Index: Tăng tốc độ truy xuất dữ liệu (data retrieval) trên một số thuộc tính (attributes) nhất định.
- Referential integrity:
  - Thuộc tính quy tắc xóa (Delete Rule) của thuộc tính tham chiếu (reference attribute)
  - Bảo vệ (Protect) - không cho phép xóa bản ghi
  - Xóa (Delete) - xóa bản ghi và thực hiện xóa theo tầng (cascades delete) toàn bộ các bản ghi đang tham chiếu đến nó

- Bỏ qua (Ignore) - không đảm bảo tính toàn vẹn tham chiếu (referential integrity)

---

Screen trong OutSystem là gì?

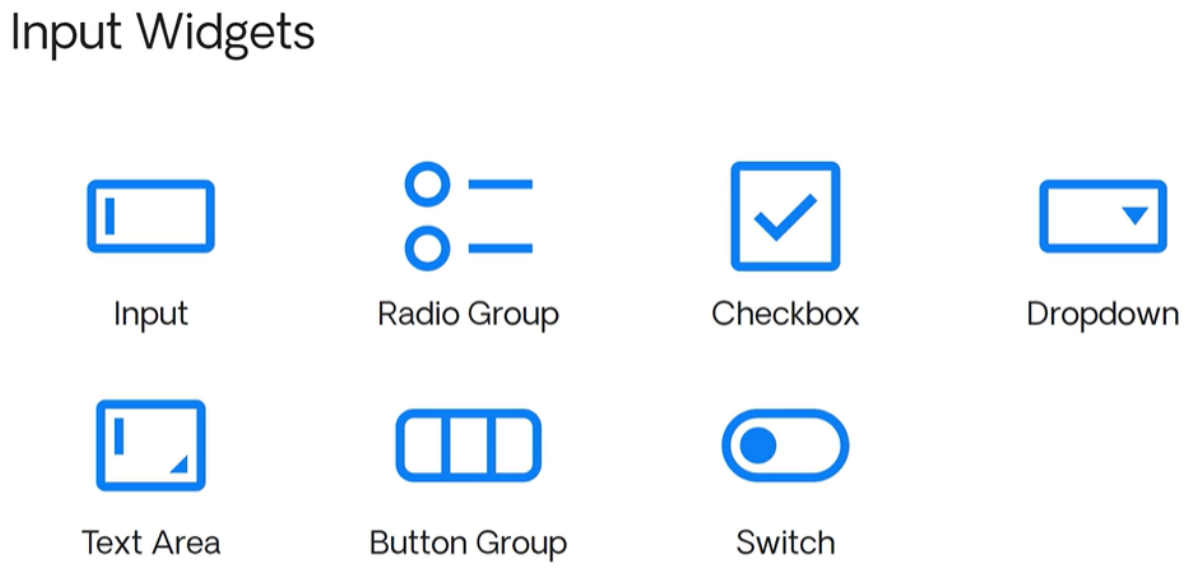
Screen là một trong những khối cấu tạo (building blocks) của một giao diện (interface), được cấu thành từ các phần tử (elements) khác để tạo nên các giao diện phong phú (rich interfaces).

---

Expression widget: Dùng để hiển thị một giá trị cụ thể (text, số, ngày tháng,...). Widget này có thuộc tính Value (không có thuộc tính Source).

Để hiển thị thông tin chi tiết của một dự án (như Name, Description), ta gán thuộc tính Value trỏ tới thuộc tính của bản ghi được lấy từ Aggregate (ví dụ: GetProjectById.Current.Project.Name).

List widget: Dùng để hiển thị một tập hợp/danh sách các phần tử. Widget này có thuộc tính Source (chứ không có thuộc tính Value).



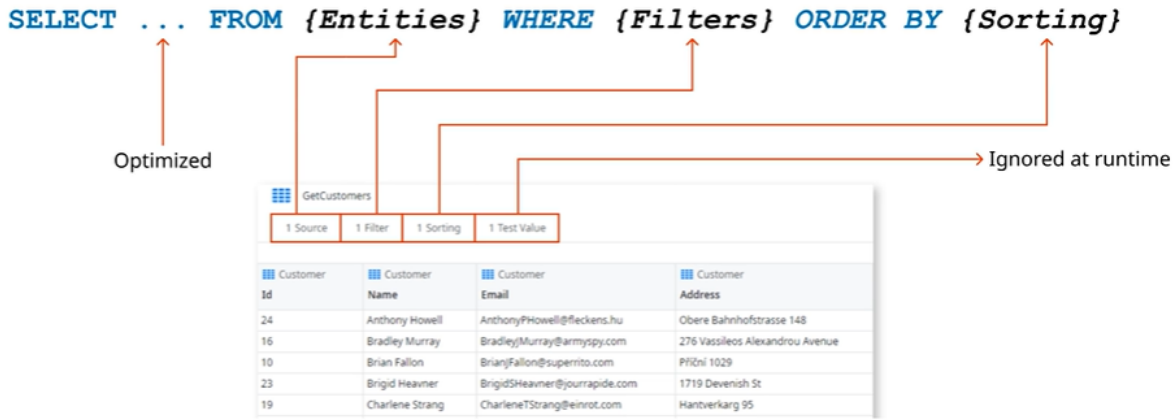

---

Aggregates là gì ?

Aggregate là một phần tử trực quan (visual element) cho phép định nghĩa một truy vấn (query) để lấy dữ liệu (fetch data) từ các thực thể (entities) đi kèm với các nguồn dữ liệu (sources), bộ lọc (filters) và quy tắc sắp xếp (sorting).

Các Aggregate cho phép chúng ta định nghĩa các truy vấn cơ sở dữ liệu (database queries) theo phương thức trực quan (visual way):

- Thêm các nguồn dữ liệu (Sources)
- Tạo các bộ lọc (Filters)
- Thiết lập các quy tắc sắp xếp (Sorting)
- Kiểm thử (Test) kết quả truy vấn




---

When is Data Fetch?

Các Aggregate và hành động dữ liệu (Data Actions) bắt đầu thực thi (executing) khi màn hình (Screen) đang trong quá trình khởi tạo (initializing). Được thực hiện một cách bất đồng bộ (asynchronously) và song song (in parallel).

Khi dữ liệu đã sẵn sàng, màn hình sẽ tự động kết xuất (renders automatically) để hiển thị dữ liệu đó lên giao diện.

---

Table Widget và List Widget

Table hoặc List đều có thuộc tính Source. Để hiển thị dữ liệu từ Aggregate, bạn cần gán thuộc tính Source này bằng danh sách kết quả đầu ra của Aggregate (ví dụ: GetUsers.List).

- Vuốt sang trái để hiển thị tùy chọn (reveal actions/options) → Full Swipe Left thành "No".
- Vuốt hết sang phải để thực thi hành động (full swipe action) → Full Swipe Right thành "Yes".

---

Form Widget

Thuộc tính Variable được khai báo ở cấp widget cha là Button Group, đại diện cho biến lưu giá trị được chọn.

Mỗi Button Group Item bên trong chỉ có thuộc tính Value (chứ không có thuộc tính Variable). Khi người dùng chọn một item, giá trị tại thuộc tính Value của item đó sẽ được gán vào Variable của Button Group.

---

Action là gì ?

Action là một phần tử (element) cho phép chúng ta định nghĩa các luồng logic (logic flows) được thực thi ở phía máy chủ (server-side) hoặc phía máy khách (client-side).

Screen Actions cũng chỉ có phạm vi (scope) trong chính màn hình đó

---

Action Flow là gì?

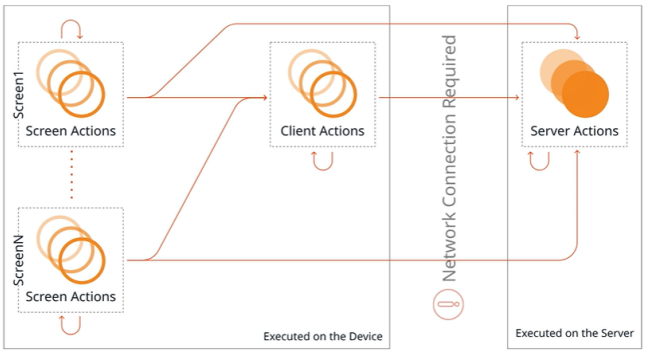
Action Flow là nơi định nghĩa một đoạn xử lý logic (piece of logic), có thể có duy nhất một Start Node

Mỗi Action Flow có thể kết thúc bằng

- End
- Điều hướng đích (Destination) (chỉ khả dụng trong các hành động màn hình - Screen Actions)
- Tải xuống (Download) (chỉ khả dụng trong các hành động màn hình - Screen Actions)

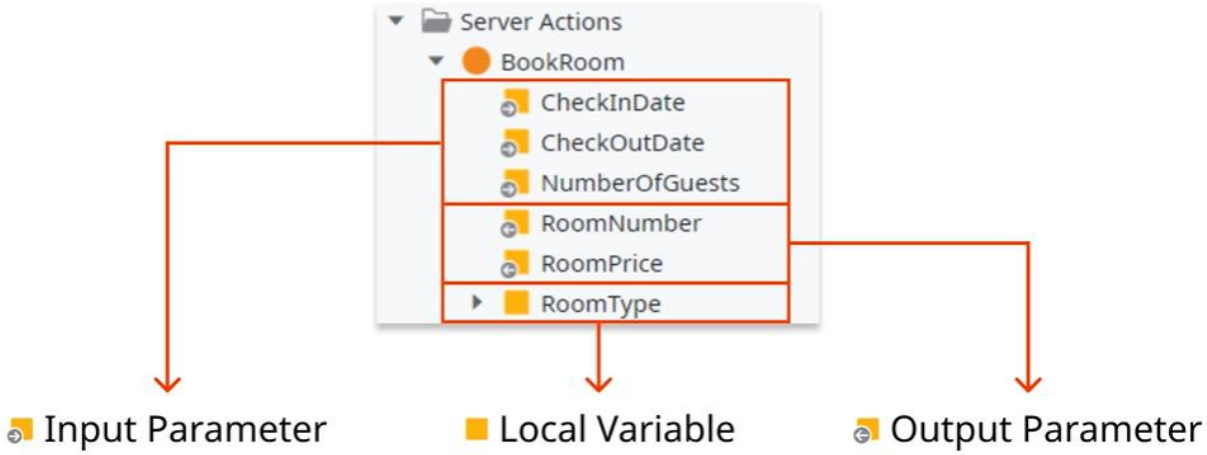
---

Code Reusability (Khả năng tái sử dụng mã nguồn) = Logic + Actions




---

Variables là gì?




Input Parameter: Truyền một giá trị (value) từ phạm vi bên ngoài (outside scope) vào bên trong.

Output Parameter: Trả về một giá trị (value) cho phạm vi bên ngoài (outside scope). Biến này tiếp tục tồn tại (continues to exist) ở phạm vi bên ngoài ngay cả khi phạm vi của phần tử cha (parent element) đã kết thúc / bị hủy.

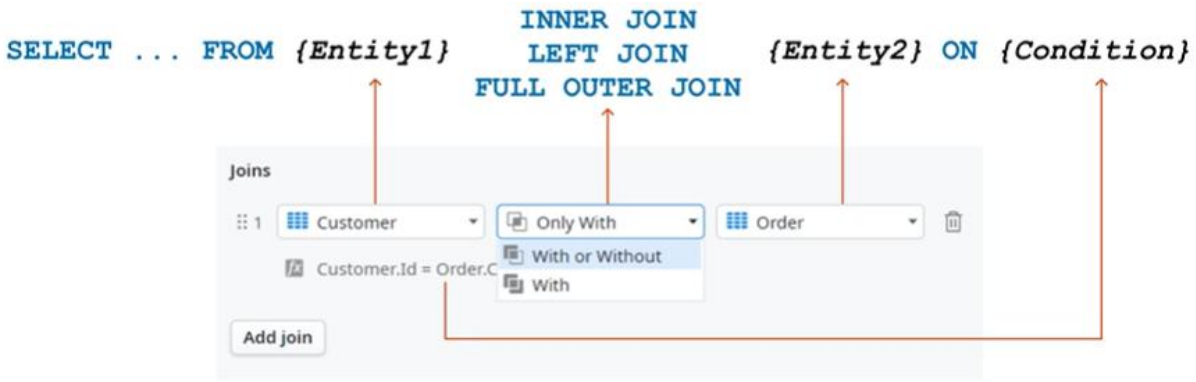
Local Variable:  Tồn tại độc quyền trong phạm vi (scope) của phần tử (element) chứa nó. Có thể được gán giá trị (assigned) và sử dụng mang tính "cục bộ" (locally) bên trong phạm vi đó. Biến (variable) này sẽ bị hủy / giải phóng (destroyed) khi tiến trình thực thi (execution) rời khỏi phạm vi của phần tử cha (parent element)

**Cả Client Action và Server Action đều hỗ trợ đầy đủ 3 loại biến**

---

Aggregate là gì?

Aggregate là một phần tử trực quan (visual element) cho phép bạn định nghĩa một truy vấn (query) nhằm lấy / nạp dữ liệu (fetch data) từ các thực thể (entities) đi kèm với các nguồn dữ liệu (sources), bộ lọc (filters) và quy tắc sắp xếp (sorting).




---

Access Control ?

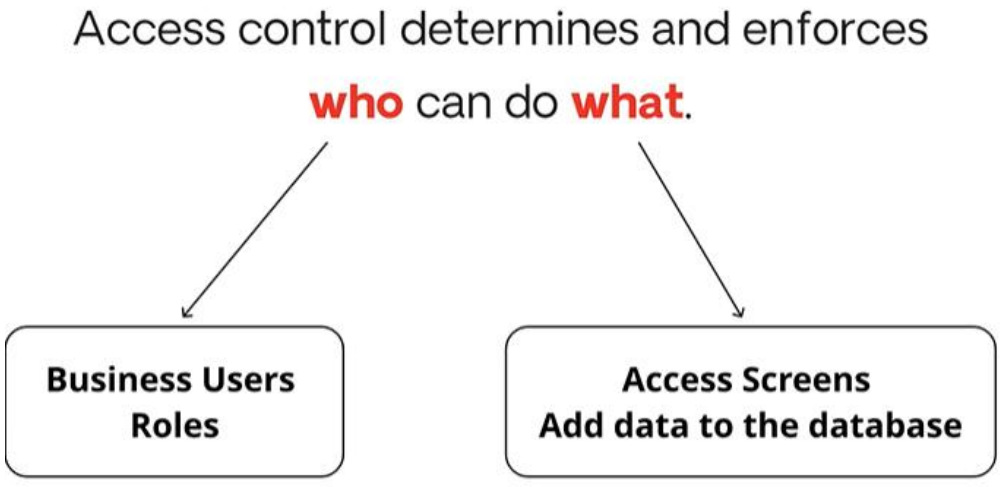
Access control bao gồm xác thực (Authentication) và phân quyền (Authorization).

Authentication:

- Xác định danh tính người đang truy cập hệ thống
- Xác minh thông tin định danh / thông tin xác thực (credentials) của người dùng

Authorization:

- Kiểm tra xem người dùng có được phép thực thi một tác vụ hay không
- Xác minh xem người dùng đã được cấp các quyền hạn yêu cầu (required permissions) hay chưa



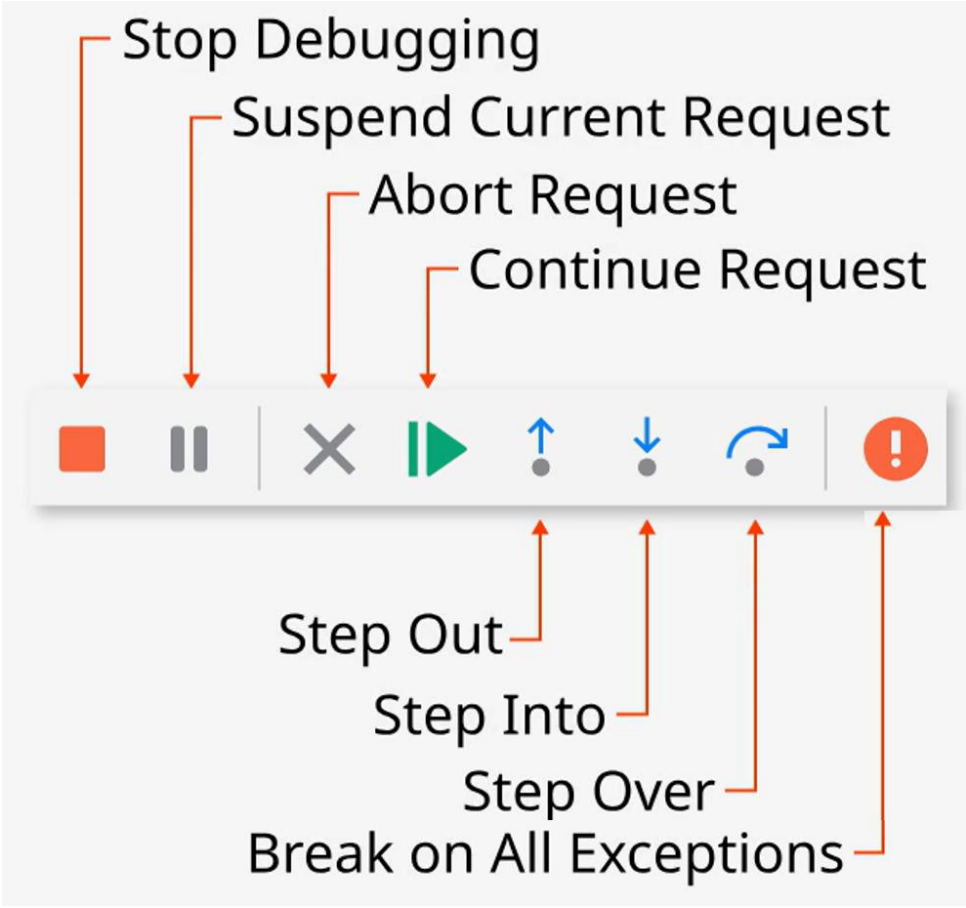
Khi bạn tạo một Role tùy biến trong OutSystems, hệ thống chỉ tự động sinh ra **3** Server Action tích hợp sẵn:

- Check Role
- Grant Role
- Revoke Role

---

Debugging là gì




---

Block là gì?




Block do not have Output Parameter

---

Client Variable là gì?

Client Variable là một biến cấu hình (configuration variable) cho phép lưu trữ thông tin có vòng đời dài (long-lived) và đặc thù theo từng người dùng (user-specific).

Bị giới hạn trong các kiểu dữ liệu cơ bản (Basic Data Types) và mã định danh thực thể (Entity Identifiers).

- Giá trị được lưu trữ ngay trong trình duyệt (browser):
  - Được giữ nguyên khi người dùng đăng nhập (Login).
  - Được khôi phục về giá trị mặc định khi người dùng đăng xuất (Logout).
- Tránh lưu trữ các thông tin nhạy cảm (sensitive) hoặc mang tính bảo mật / tuyệt mật (confidential information).

A good use case for a Client Variable: User Name

---

Site Properties là gì?

Site Property là một biến cấu hình phía máy chủ (server-side configuration variable) cho phép lưu trữ thông tin trên phạm vi toàn ứng dụng (application-wide)

---

Cú pháp viết lệnh SQL trong OutSystems

- Tên Entity: {Enity}

- Tên Attribute: [Attribute]

Ví dụ: {EmployeePicture}.[Picture]

---

Css trong OutSystems

Static

- Design-time
- Simpler
- Live Preview

Dynamic

- Run-time
- More Flexible
- Limited Preview

---

JavaScript cho các ứng dụng tương tác (Reactive Applications)

Trong OutSystem bạn có thể thực thi JavaScript tại các vị trí:

- Client Action trong các Screen, Block (Sử dụng trong các nút Logic)
- Global Client Action trong Logic Tab
- Các Module

Add và Run JavaScript trong Screen Client Action

- Tạo một Screen Client Action
- Thiết lập một Action để Handler :
    - OnClick của một Button hoặc Link
    - OnChange của Dropdown hoặc Input
- Kéo thả JavaScript Node vào Flow và viết Sript

Add và Run JavaScript trong Global Client Action

- Tạo một Global Client Action tại Logic Tab
- Kéo thả JavaScript Node vào Flow và viết Sript
- Gọi Global Client Action này bên trong bất kỳ hành động nào khác được thực thi trong thời gian chạy (executes in runtime) để tái sử dụng logic JavaScript trên toàn ứng dụng.

Keywords JavaScript

- $parameters

```
var num;
$parameters.outputNumberParam = $parameters.inputNumberParam * num;
```

- $actions

```
var num, result;
result = $actions.ClosestPrimeNumber(num).Prime;
```

- $public

```
$public.Navigation.navigaTo ($parameters.screenName);
```

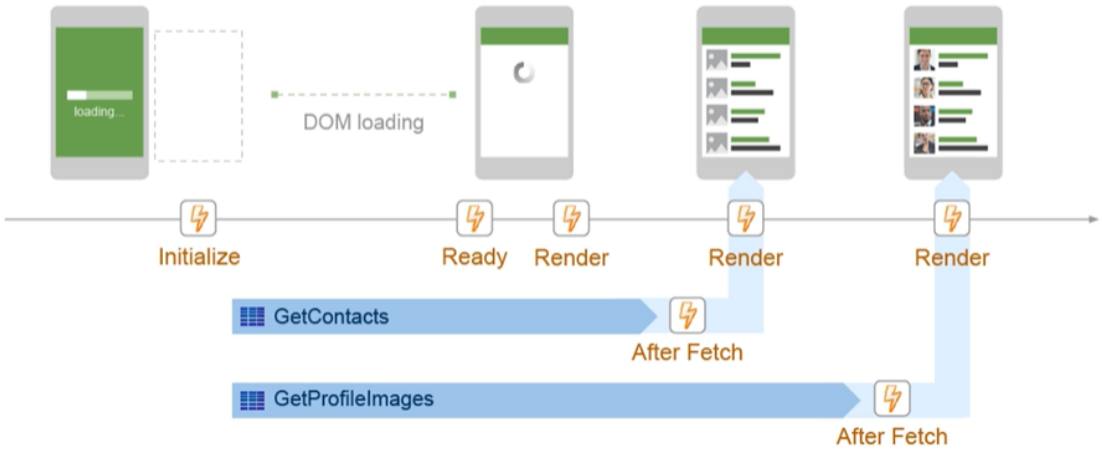
- $role

```
$public.Security.checkIfCurrentUserHasRole($roles.Manager);
```

# Widget Events

- Input
  - on**change**
- Focus
  - on**focus**
  - on**blur**
- Touch
  - on**touchstart**
  - on**touchend**
  - on**touchmove**
- Keyboard
  - on**keydown**
  - on**keypress**
  - on**keyup**
- Mouse
  - on**click**
  - on**doubleclick**

## Screen Life cycle



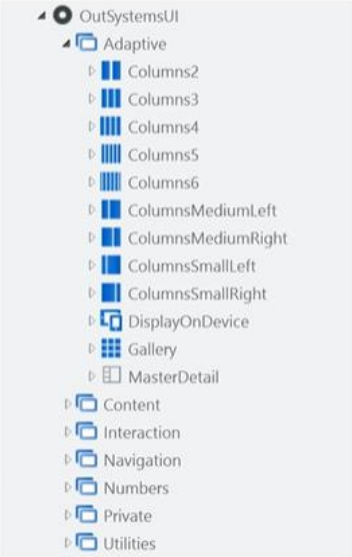

What is Pattern?

Một Pattern (mẫu giao diện / mẫu thành phần) là một tập hợp các phần tử (elements) kết hợp với nhau nhằm tạo ra một khối giao diện tái sử dụng (reusable block).

Các đặc điểm:

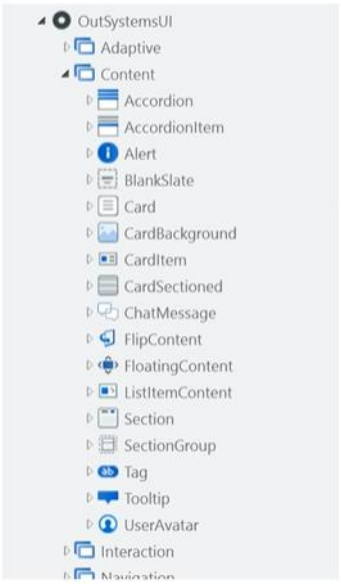
- Hiện thực hóa giao diện và tương tác chuẩn xác
- Có thể tùy biến linh hoạt (Customizable)
- Có khả năng mở rộng cao (Extensible)

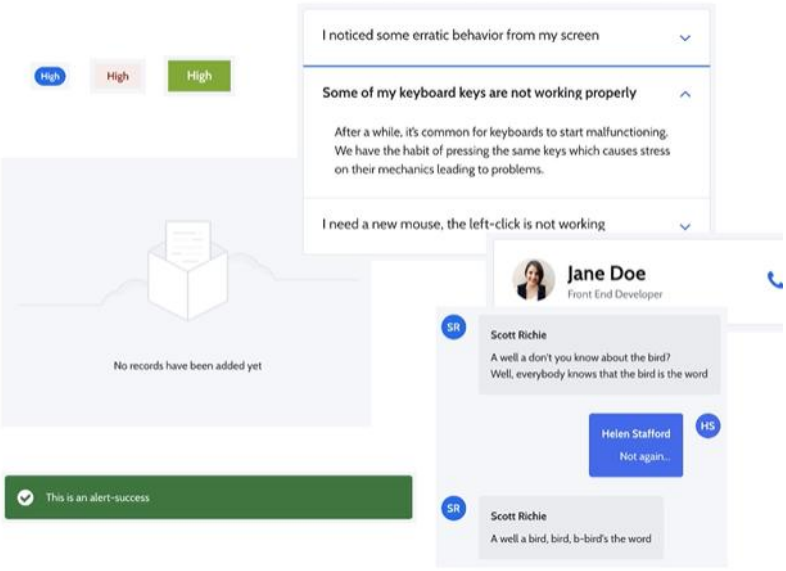
# Adaptive patterns



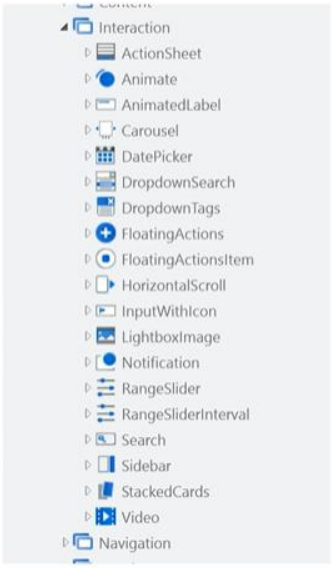

- Organize generic content areas on the screen
  - Responsive
  - Configurable per device type
- Common configuration options
  - Gutter between content areas
  - Device-oriented breaks

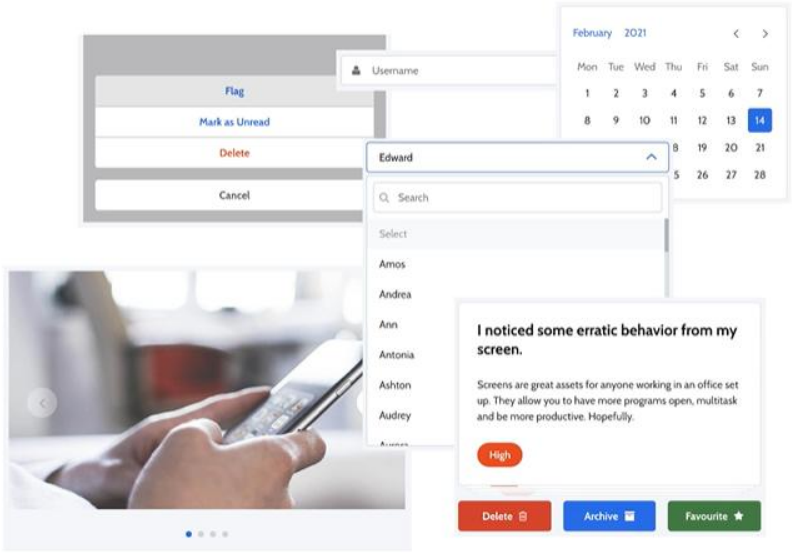
# Content patterns



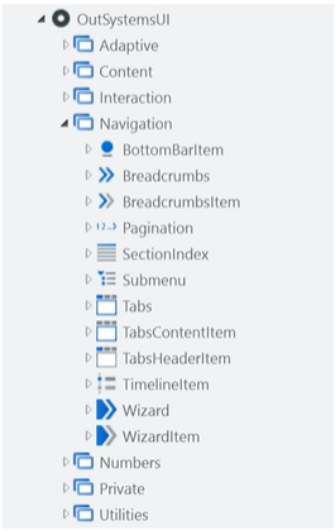



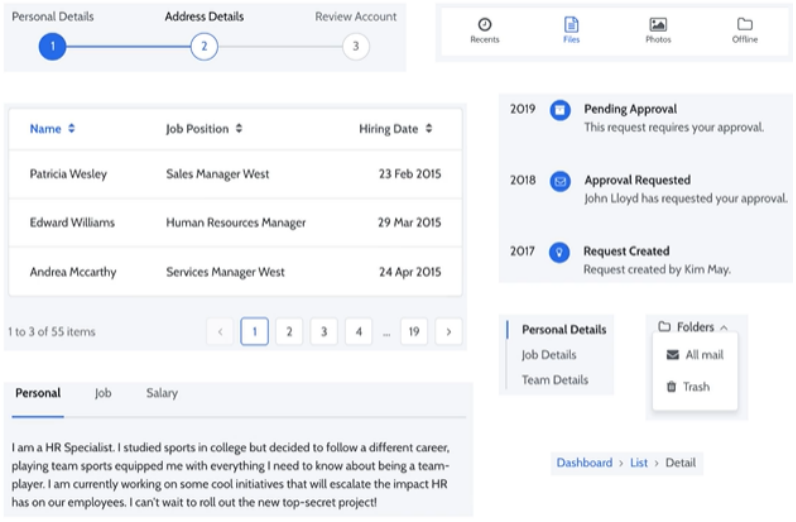
# Interaction patterns



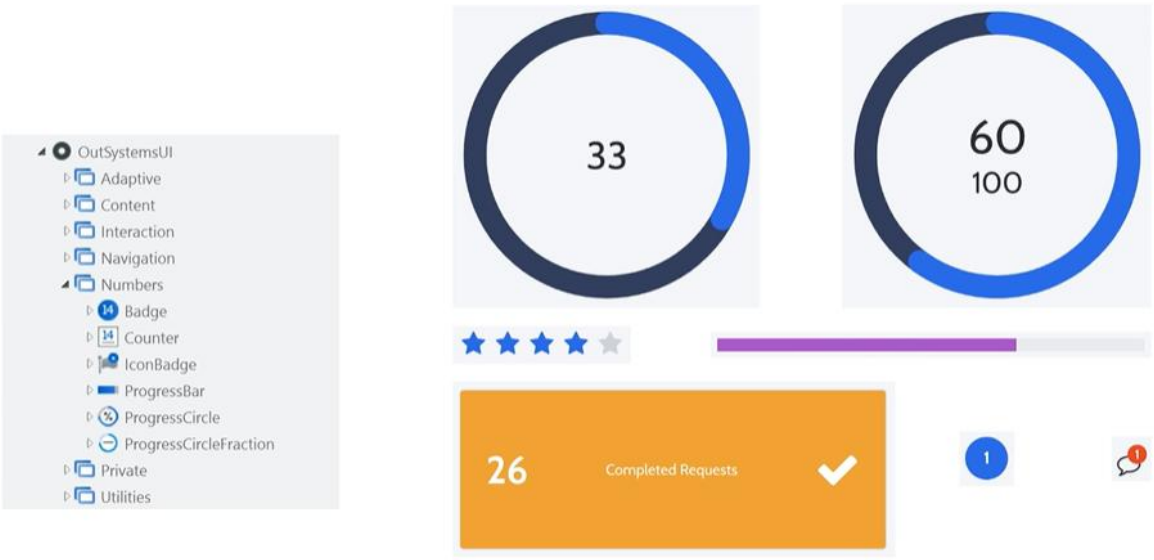




# Navigation patterns

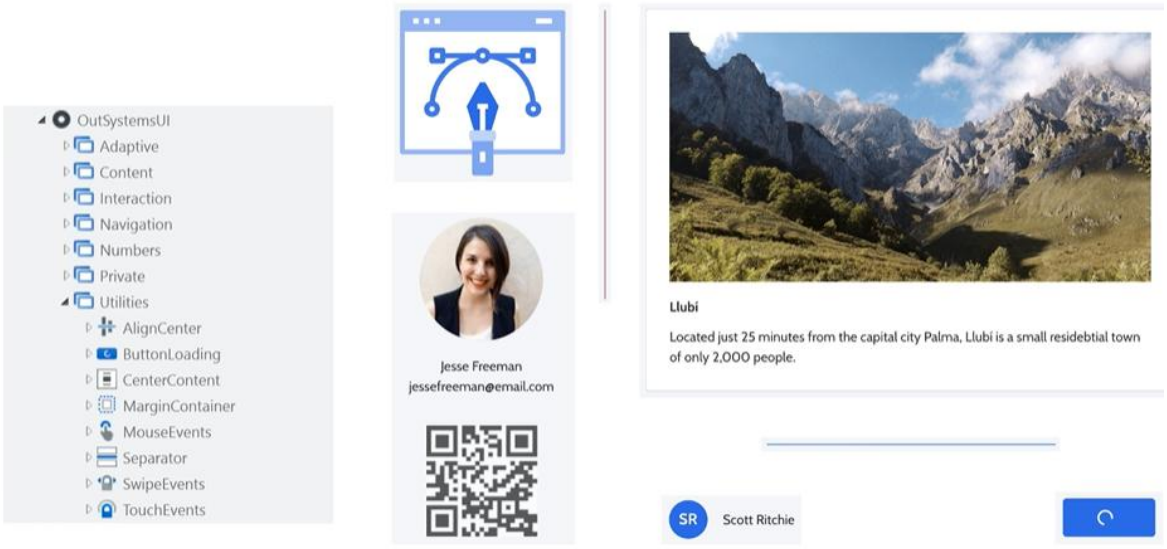


## Numbers patterns




## Utilities patterns




---

What is Accordion?

Accordion là Mẫu danh sách xếp tầng / Khung thu gọn - mở rộng. một mẫu giao diện người dùng (UI Pattern) dạng danh sách dọc cho phép hiển thị và ẩn nội dung theo từng phần độc lập.

Mẫu thiết kế này giúp tối ưu hóa không gian hiển thị trên giao diện bằng cách cho phép người dùng chủ động bung mở (expand) hoặc thu gọn (collapse) các khối thông tin khi cần thiết.

---

# Reactive applications

- The structure for reactive applications

